Материалы кафедры ММП факультета ВМК МГУ. Введение в глубокое обучение.

Занятие 12. Решение дифференциальных уравнений с помощью нейронных сетей. PINN, DeepONet, FNO

Занятие провел: Иванов Егор (@e1vanov)

Материал составил: Иванов Егор (@e1vanov)

Москва, Весенний семестр 2026

Источники:

- Статья 1 по **PINN**. Решение ДУ: Maziar Raissi et al., 2017, примерно 2200 цитирований
- Статья 2 по **PINN**. Обратные задачи: Maziar Raissi et al., 2017, примерно 2200 цитирований
- **DeepONet**: Lu Lu et al., 2019, примерно 1200 цитирований
- Neural Operator (**FNO, LNO, ...**): Nikola Kovachki et al., 2021, примерно 2000 цитирований
- Теорема об универсальной аппроксимации операторов. Chen, Chen, 1995, примерно 1800 цитирований
- Обучение операторов. Обзор: Nicolas Boullé, Alex Townsend, 2023, примерно 140 цитирований

Обсудим, как можно научить нейронную сеть физике и как с помощью этого можно

построить альтернативу классическим численным солверам ОДУ и обратных задач. Кроме того, на прошлых лекциях вы узнали, что нейронные сети являются универсальными аппроксиматорами функций. В этот раз мы повысим ставки и поговорим о том, могут ли нейронные сети аппроксимировать отображения между бесконечномерными пространствами - операторы, например, отображающие функции, задающие ДУ, в его решение.

Мотивация

Рассмотрим задачу бинарной классификации изображений кошек и собак.

Вопрос (на повторение): Как можно ее решать?

Ответ:

Составим решающее правило как композицию нейронных сетей следующего вида:

- **Извлечение признаков**: сверточная нейронная сеть (CNN)
- **Обработка признаков**:

полносвязная сеть (MLP)

- **Функция потерь**:

бинарная кросс-энтропия (BCE)

Вопрос. Используется ли в предложенном решении тот факт, что в датасете именно кошки и собаки? Изменится ли подход, если это будут автомобили и самолеты?

Ответ:

Нет, не изменится.

Для многих прикладных задач априорные знания плохо поддаются математической формализации, и мы надеемся, что сеть в процессе оптимизации *"сама разберется"*. Из-за этого (или благодаря этому) естественный отбор при эволюции методов прошли **самые универсальные** слои, архитектуры, методы оптимизации и т.д.

Ключевая идея

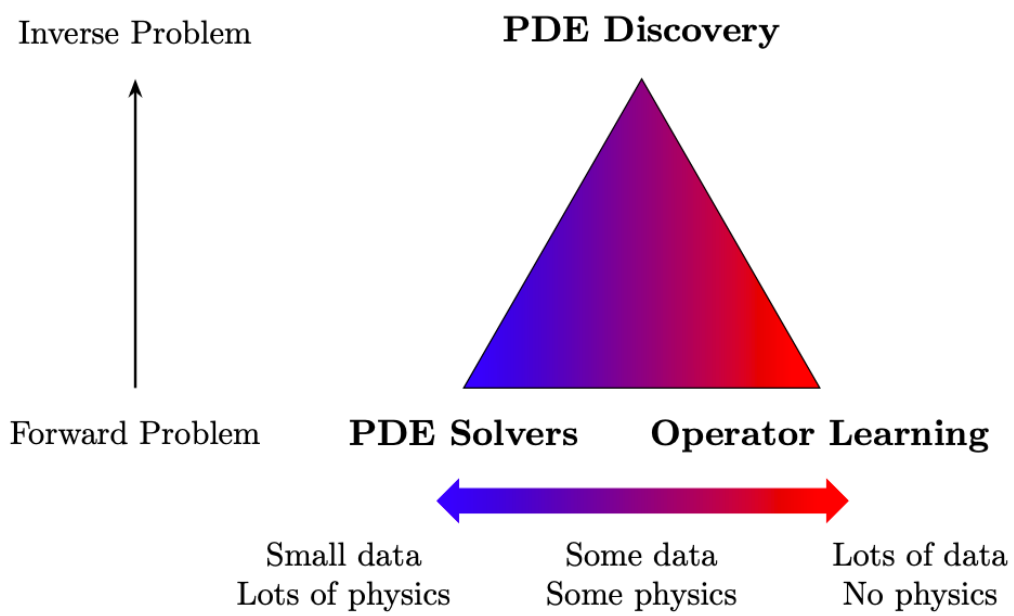
Что, если применить универсальные подходы для задач с математически формализованным априорным знанием, каковыми являются **физические** задачи?

Возьмем **лучшее** от обоих подходов:

- от нейронных сетей – универсальность,
- от физической постановки – априорные знания.

Постановки задач

Опр. Scientific **ML** (SciML) – область глубокого обучения, изучающая применение нейросетей для решения физических задач.



Источник

Существует несколько вариантов постановки задачи, они отличаются между собой соотношением количества известной "физики" и данных:

- **Решение уравнений (УЧП, ОДУ)**. Предположим, вам известны законы, которым подчиняется некоторая физическая система (пусть, например, они выражены в форме дифференциальных уравнений). Кроме того, имеется некоторое (обычно небольшое) количество измерений этой системы. Цель: построить модель этой физической системы.

- **Обратные задачи (восстановление параметров)**. У нас есть некоторое количество измерений, но физика процесса известна не полностью. Цель: восполнить этот недостаток на основе данных.

- **Operator Learning**. Физическая задача порождает оператор, действующий между бесконечномерными пространствами. Цель: обучить нейронную сеть, аппроксимирующую этот оператор. В качестве примера, имеющего прикладную

ценность, можно привести оператор, ставящий в соответствие функциям, задающим ДУ, его решение.

Physics-Informed Neural Networks (PINNs)

Первой успешной попыткой применения нейронных сетей для решения физических задач стали Physics-Informed Neural Networks (PINNs).

Рассмотрим следующую постановку задачи.

Имеется одномерный $x \in (0, 1)$ стержень, который остывает в течение интервала времени $t \in (0, 1)$ при некотором изначальном распределении температуры $\varphi(x)$. Обозначим за $u(t, x)$ температуру стержня в момент времени t в координате x.

В нашем распоряжении имеется набор измерений температуры стержня с датчиков – обучающая выборка:

$$\{(x_n, t_n), u_n\}, n \in 1, \ldots, N$$

Требуется определить температуру стержня в других точках – на тестовой выборке.

Предложим **наивное** решение с использованием нейронных сетей.

Действительно, искомая функция u это отображение $\mathbb{R}^2 \to \mathbb{R}^1$, которое можно параметризовать нейронной сетью $f_\theta(t, x)$, например, полносвязной. Для определения параметров модели будем минимизировать MSE с данными наблюдений:

$$\theta^* = \operatorname{argmin} \frac{1}{N} \sum_{n=1}^{N} |u_n - f_\theta(t_n, x_n)|^2$$

Вопрос: Какие проблемы могут возникнуть при решении поставленной задачи таким способом?

Ответ:

Предложенный подход будет подвержен *переобучению*. Действительно, если данных наблюдений не слишком много и/или в них присутствует шум, то обучение нейронной сети с легкостью может обеспечить нулевые потери на тренировочной выборке с **нефизичным** решением вне нее.

Physics-informed Loss

Будем бороться с переобучением с помощью регуляризатора, основанного на физике задачи. Действительно, функций, интерполирующих данные наблюдений, много,

поэтому разумно наложить дополнительные ограничения. Выпишем уравнение теплопроводности, которому подчиняется распределение температуры в стержне.

$$\begin{cases} u_t - \lambda u_{xx} = 0 \\ u(t,0) = u(t,1) = 0 \\ u(0,x) = \varphi(x) \end{cases}$$

- В случае, если λ известен, мы находимся в постановке решения УЧП – обучаем функцию, которая описывает наблюдения и соответствует физическому закону во всей области.
- Если же λ не известен, то задача становится обратной: мы доуточняем физику на основе данных, добавляя к оптимизируемым переменным неизвестные физические постоянные.

Под соответствием физике мы понимаем то, что функция является решением задачи, то есть удовлетворяет дифференциальному уравнению, граничным и начальным условиям. Самый простой способ это сделать – **внести эти требования в функцию потерь.**

$$\mathsf{L}_{\text{total}} = \mathsf{L}_{\text{data}} + \alpha_1 \mathsf{L}_{\text{de}} + \alpha_2 \mathsf{L}_{\text{boundary}} + \alpha_3 \mathsf{L}_{\text{initial}},$$

где, например,

$$\mathsf{L}_{\text{de}} = \frac{1}{M} \sum_{i=1}^{M} \left| \partial_t \text{NN}(t_i', x_i') - \lambda \, \partial_{xx} \text{NN}(t_i', x_i') \right|^2$$

Вопрос: Почему на такую функцию потерь можно обучать нейронную сеть?

Ответ:

Функция, задаваемая нейронной сетью дифференцируема как по параметрам, так и по входам, причём производные по входам – это так же некоторые дифференцируемые функции от параметров.

Единственное, что может смутить – потребление памяти. Подсчет третьих (или даже более высокого порядка) производных может показаться жутким, что действительно так в случае гигантских сеток, но в физических задачах размерности низкие, а сами сетки – редко сложнее сверточных.

Вопрос: В каких точках считать регуляризатор в функции потерь?

Ответ:

Версия №0: в тех же, в которых известны данные наблюдений, хотя на самом деле верно более смелое предположение – **в любых**.

Действительно, эти точки авторы PINN'ов предложили называть *виртуальными*, что точно отражает суть их использования.

Будем использовать составную функцию потерь:

$$\mathsf{L}_{\text{total}} = \mathsf{L}_{\text{data}} + \alpha_1 \mathsf{L}^{\text{de}}_{\text{virtual}} + \alpha_2 \mathsf{L}^{\text{boundary}}_{\text{virtual}} + \alpha_3 \mathsf{L}^{\text{initial}}_{\text{virtual}},$$

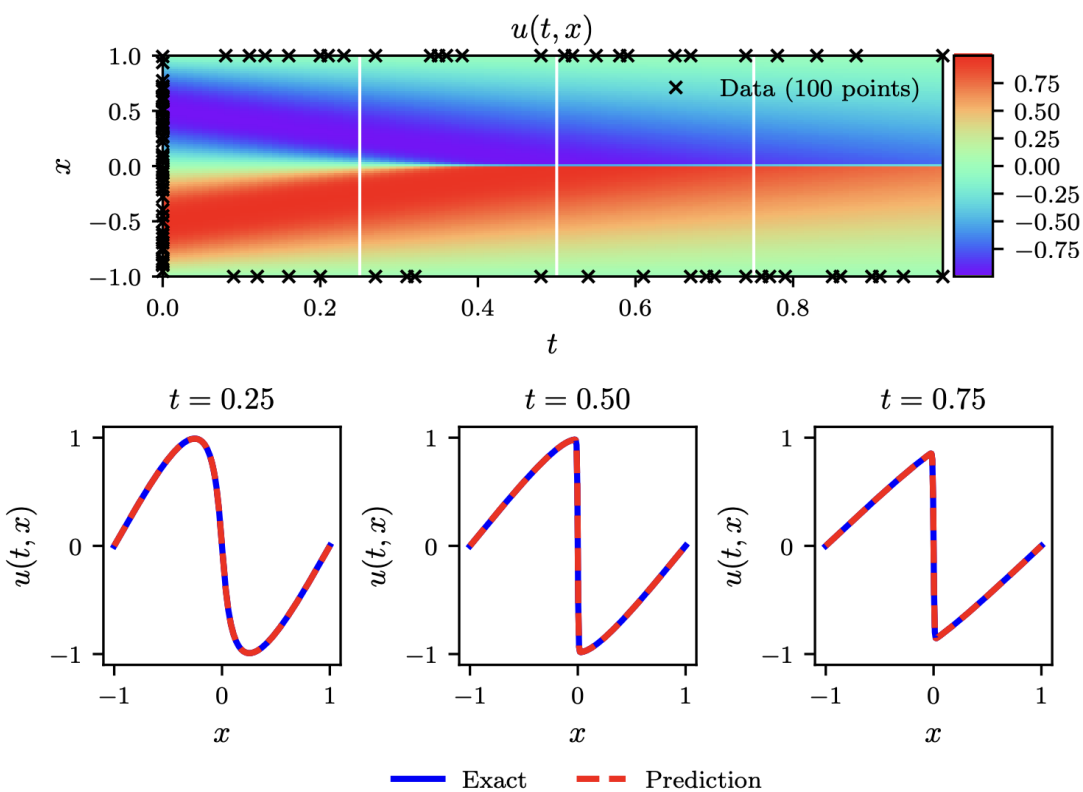
где, например,

$$\mathsf{L}^{\text{de}}_{\text{virtual}} = \frac{1}{N_{\text{virtual}}} \sum_{i=1}^{N_{\text{virtual}}} \left| \partial_t \text{NN}(t_i, x_i) - \lambda \, \partial_{xx} \text{NN}(t_i, x_i) \right|^2$$

Виртуальные точки являются очень полезной степенью свободы. Например, их можно выбирать новыми для каждого батча / эпохи разными способами – случайной инициализацией или основываясь на априорных знаниях о виде функции.

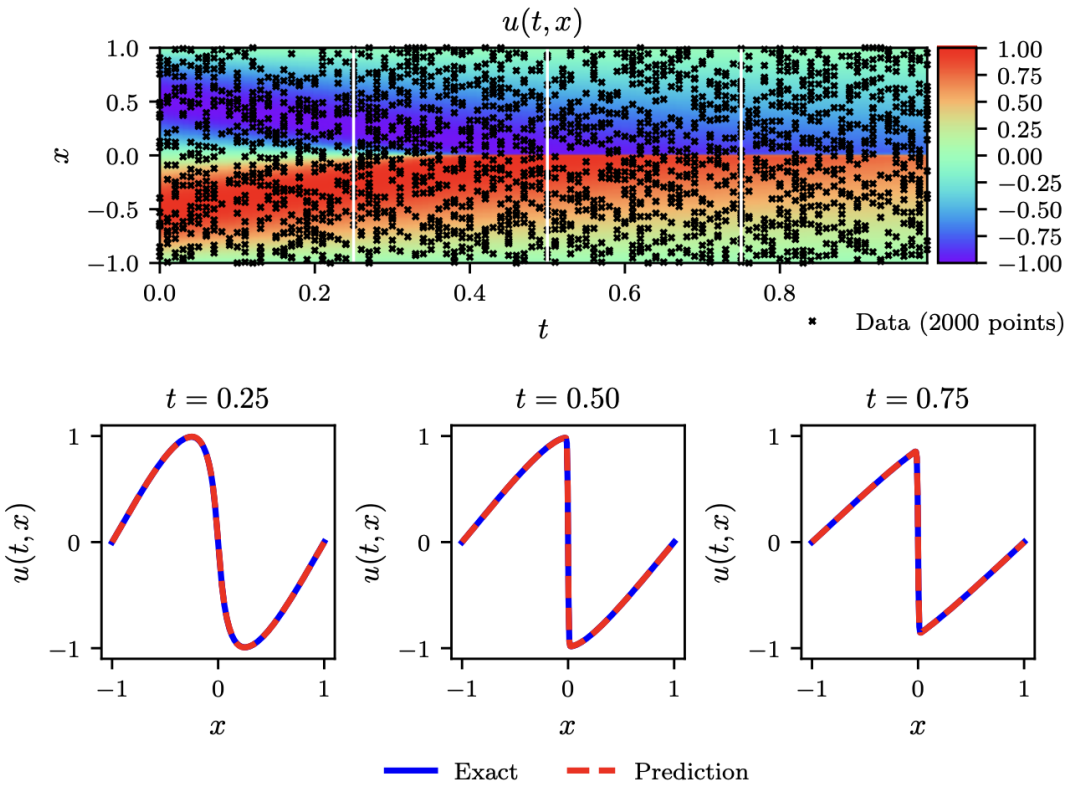
Пример из статьи. Решение ДУ

$$\begin{aligned}
&u_t + u u_x - (0.01/\pi) u_{xx} = 0, \quad x \in [-1, 1], \quad t \in [0, 1], \\
&u(0, x) = -\sin(\pi x), \\
&u(t, -1) = u(t, 1) = 0.
\end{aligned}$$



Источник

Пример из статьи. Обратная задача



Correct PDE	$u_t + uu_x - 0.0031831u_{xx} = 0$
Identified PDE (clean data)	$u_t + 0.99915uu_x - 0.0031794u_{xx} = 0$
Identified PDE (1% noise)	$u_t + 1.00042uu_x - 0.0032098u_{xx} = 0$

Источник

Operator Learning (DeepONet, FNO)

Повысим сложность: можно ли "выучить" целую задачу (оператор), а не конкретное ДУ.

Напомним постановку задачи классического обучения с учителем: имеется набор прецедентов $\{(x_i, y_i)\}$ действия некоторого отображения $f : \mathsf{X} \to \mathsf{Y}$. Требуется восстановить это отображение на основе данных наблюдений.

До этого пространства X и Y были конечномерными. Что, если попробовать обобщить эту постановку на бесконечномерный случай? Иными словами, пример действия отображения (оператора) теперь – пара функций (u, v). Рассмотрим следующие примеры.

- **Пример 1.** УЧП

$$-\nabla \cdot (a(\mathbf{x})\nabla u(\mathbf{x})) = f(\mathbf{x}),\ \mathbf{x} \in D$$
$$u(\mathbf{x}) = 0,\ \mathbf{x} \in \partial D$$

Функции a и f подаются на вход нейронной сети, выходом которой является функция u – решение УЧП.

- **Пример 2.** Динамическая система

$$\frac{dx}{dt} = u(t),\ t \in [0, 1]$$
$$x(0) = x_0$$

Функция u подается на вход нейронной сети, выходом является функция x.

Вопрос: Что вы можете предложить в качестве решения?

Ответ:

В качестве идеи №0 можно отождествить функцию с ее конечной дискретизацией и обучать нейронную сеть отображать сетку значений входа в сетку значений выхода.

Предложенное решение безусловно рабочее при определенных условиях, например, в случае, если известно, что дискретизация не будет меняться. Однако перед нами стоит цель аппроксимировать именно оператор.

Кроме того, под тем, что нейронная сеть оперирует над функциями, мы понимаем, что для этих функций возможно вычисление значения в произвольной точке. Можно возразить, что это не проблема, поскольку эти значения можно, например, интерполировать. Это действительно так.

Более сложным и интересным свойством, которое мы ожидаем от нейронного оператора, является "независимость" от дискретизации. Под независимостью мы понимаем то, что нейронная сеть способна обрабатывать функции на произвольной дискретизации и, более того, уточнять свое предсказание при ее измельчении.

DeepONet

Теорема об универсальном аппроксиматоре

Теорема (Цыбенко; 1989).

Пусть $\phi(\xi) = 1/(1 + \exp\{-\xi\})$ - сигмоидная функция. Тогда для любой непрерывной функции $f : X \to \mathbb{R}$, где X - компакт в $\mathbb{R}^n$ и $\varepsilon > 0$ существуют векторы $\mathbf{w}_1, \ldots, \mathbf{w}_N, \boldsymbol{\alpha}, \boldsymbol{\theta}$ такие, что неравенство

$$\left| f(\mathbf{x}) - \sum_{i=1}^{N} \alpha_i \phi(\langle \mathbf{x}, \mathbf{w}_i \rangle + \theta_i) \right| < \varepsilon$$

справедливо всюду в X.

Иными словами, для наперед заданных функции и погрешности существует *достаточно широкая* полносвязная нейронная сеть с 2 слоями, обеспечивающая нужную точность.

Вопрос (на повторение): Данная теорема является не более чем *оправданием* использования полносвязных сетей для решения задач глубокого обучения. Почему?

Ответ:

Действительно, теорема утверждает, что класс полносвязных сетей может с любой точностью приблизить любую непрерывную функцию. Ответа на вопрос, как такой аппроксиматор искать и сойдётся ли метод оптимизации к нему по обучающей выборке, теорема не дает.

Теорема об универсальном аппроксиматоре. Операторный случай

Оказывается, что схожий результат есть и в операторном случае:

Теорема (Chen, Chen; 1995).

Пусть σ - непрерывная неполиномиальная функция, X – банахово пространство, K_1 – компакт в X, K_2 – компакт в $\mathbb{R}^d$, V – компакт в $C(K_1)$, $G : V \to C(K_2)$ – нелинейный непрерывный оператор. Тогда для любого числа ε существуют числа n, p, m, константы $c_i^k, \xi_{ij}^k, \theta_i^k, \zeta_k \in R$ и векторы $\mathbf{w}_k \in \mathbb{R}^d$, $\mathbf{x}_j \in K_1$, $i = 1, \ldots, n,\ k = 1, \ldots, p,\ j = 1, \ldots, m$ такие, что неравество

$$\left|G(u)(\mathbf{y}) - \sum_{k=1}^{p} \sum_{i=1}^{n} c_i^k \sigma \left(\sum_{j=1}^{m} \xi_{ij}^k u(\mathbf{x}_j) + \theta_i^k \right) \sigma(\langle \mathbf{w}_k, \mathbf{y} \rangle + \zeta_k) \right| < \varepsilon$$

справедливо для любых $u \in V$ и $\mathbf{y} \in K_2$.

Удивительный факт. Статья, в которой была доказана эта теорема, завершается рассуждениями авторов о том, что такой вид может позволить обучать нейронные сети для аппроксимации операторов. Реализация этой идеи случилась лишь спустя 25 лет в статье по DeepONet.

Если победить страх большого количества обозначений, можно обратить внимание на то, что универсальный аппроксиматор, предложенный в теореме выше, представляет собой скалярное произведение двух полносвязных сетей:

- **Branch Net:** сеть, принимающая

на вход значения функции в фиксированном наборе точек $\{\mathbf{x}_j\}$

- **Trunk net:** сеть, принимающая на вход

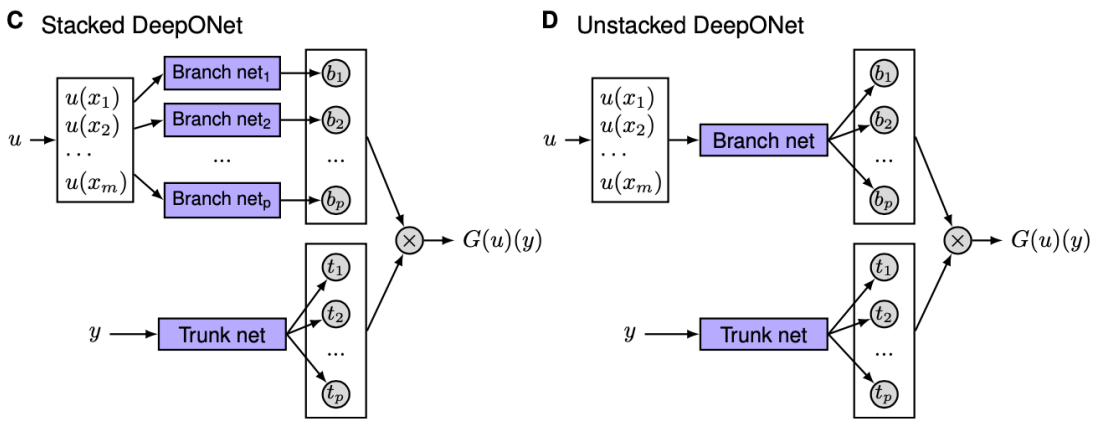
точку $\mathbf{y}$, в которой нужно вычислить результат действия оператора.

Кроме того, можно увидеть и идею, связанную с разделением переменных:

$$G(u)(\mathbf{y}) \approx \hat{G}(u(\mathbf{x}_1), \ldots, u(\mathbf{x}_m), \mathbf{y}) = \langle \mathrm{branch}(u(\mathbf{x}_1), \ldots, u(\mathbf{x}_m)), \mathrm{trunk}(\mathbf{y}) \rangle$$

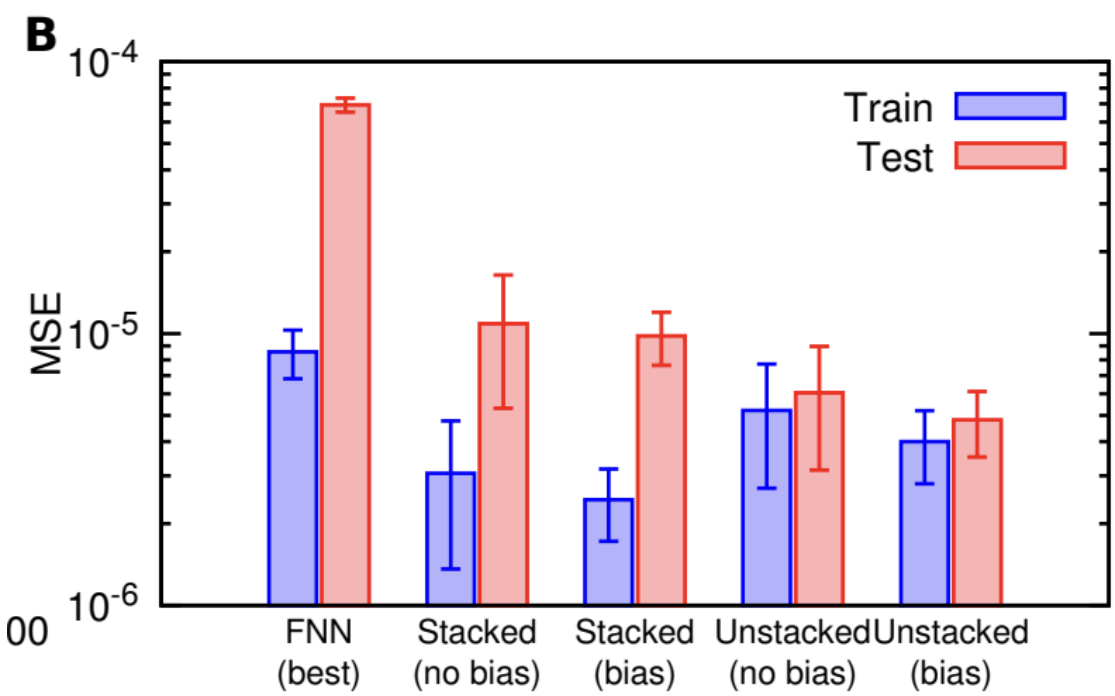
Задача аппроксимации результата действия оператора (функции) в точке разбивается на две: извлечение информации о самой функции и об интересующей точке.

На основе утверждения теоремы авторы статьи предложили следующие архитектуры:



Источник

`Пример из статьи. Уменьшение переобучения`



Источник

Вопрос: Удовлетворяет ли DeepONet тем свойствам, которые мы требуем от нейронного оператора?

Ответ:

Результат действия оператора может быть вычислен в произвольной точке (без использования интерполяции) – она подается на вход. Набор узлов фиксирован, поэтому изменить (и тем более измельчить) дискретизацию входа нельзя.

Neural Operators. FNO

Мотивация

Напомним функциональный вид полносвязных сетей:

$$\mathrm{NN}(\mathbf{x}) = \sigma\left(\mathbf{A}_L(\cdots\sigma(\mathbf{A}_1\mathbf{x} + \mathbf{b}_1)\cdots) + \mathbf{b}_L\right)$$

Аппроксиматор строится как поочередная композиция линейных и нелинейных отображений. Используя упомянутые выше теоремы, а также прикладной опыт использования полносвязных и сверточных нейронных сетей, можно предложить их обобщение на бесконечномерный случай.

- В качестве *линейного* отображения предлагается

использовать интеграл с обучаемым ядром. Это отображение инвариантно относительно дискретизации и сходится к истинному значению интеграла при ее измельчении.

$$\mathsf{K}_i(u_i)(\mathbf{x}) = \int_{\Omega_i} K_i(\mathbf{x}, \mathbf{y}) u_i(\mathbf{y}) d\mathbf{y}$$

- В качестве *смещения* так же предлагается использовать функцию:

$$\mathsf{B}_i(u_i)(\mathbf{x}) = u_i(\mathbf{x}) + b_i(\mathbf{x})$$

- В качестве *нелинейных* отображений (активаций)

предлагается использовать нелинейные функции σ, применяемые поточечно. Например, часто используемые в глубоком обучении ReLU, $\tanh$ и т.д.

Теорема об универсальном аппроксиматоре

Теорема (2022). Пусть A, U – банаховы пространства функций, $\mathsf{G}^\dagger : \mathsf{A} \to \mathsf{U}$ – нелинейный оператор. При определенных предположениях на все упомянутые выше объекты, если положить ядра и смещения полносвязными нейронными сетями, для любого компакта $K \subset \mathsf{A}$ и $0 < \varepsilon \leq 1$ существует $N \in \mathbb{N}$ и нейронный оператор NO_N такой, что

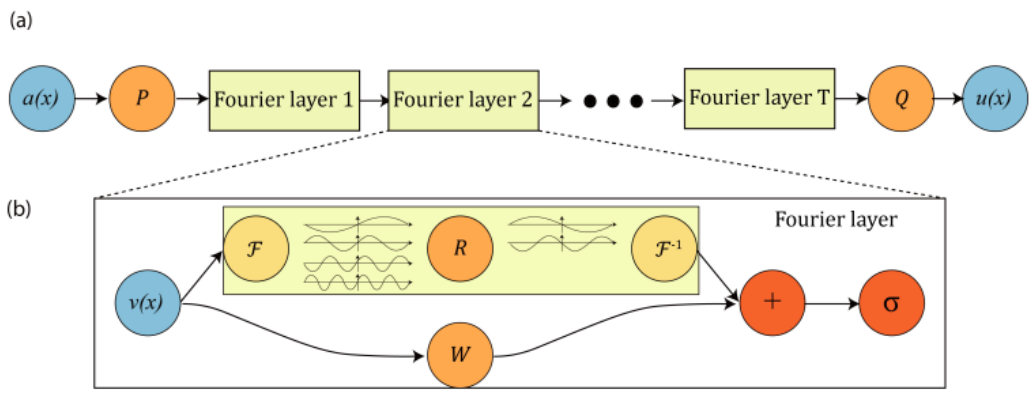
$$\sup_{a \in K} \|\mathrm{NO}_N(a) - \mathsf{G}^\dagger(a)\|_{\mathsf{U}} \leq \varepsilon$$

Архитектура нейронных операторов

Архитектуры нейронных операторов отличаются друг от друга способом параметризации и вычисления интегральных операторов.

Наиболее известный подход – *Fourier Neural Operator (FNO)*, основанный на использовании операции свертки, т.е.

$$\mathsf{K}_i(u_i)(\mathbf{x}) = \int_{\Omega_i} K_i(\mathbf{x}, \mathbf{y}) u_i(\mathbf{y}) d\mathbf{y} = \int_{\Omega_i} K_i(\mathbf{x} - \mathbf{y}) u_i(\mathbf{y}) d\mathbf{y} = \mathsf{F}^{-1}\mathsf{F}(K_i)\mathsf{F}(u_i)$$



Источник

Пройдем по слоям и выпишем размерности возникающих тензоров:

- Дискретизация функций на входе: (T)
- Поточечный подъем размерности: (T, H)
- Преобразование Фурье вдоль временной оси: (F, H)
- Обрезаем высокие частоты: (Fr, H)
- Для каждой частоты применяем линейное преобразование: (Fr, H)
- Дополняем нулями старшие гармоники: (F, H)
- Делаем обратное преобразование Фурье: (T, H)
- Поточечное понижение размерности: (T)

Вопрос: Удовлетворяет ли FNO тем свойствам, которые мы требуем от нейронного оператора?

Ответ:

Да, но с оговорками. Вычисление в произвольной точке осуществляется за счет интерполяции. Дискретизация произвольная, но регулярная для того, чтобы применять

Фурье.

Пример из статьи

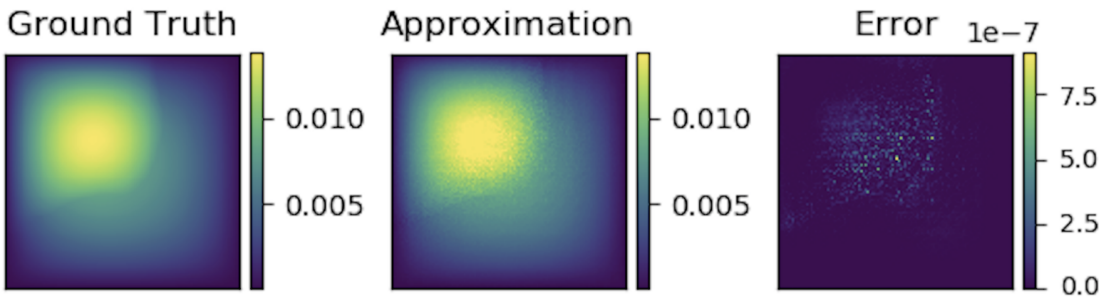


Figure 9: Darcy, trained on 16×16, tested on 241×241

Источник

Приложения нейронных операторов

Несмотря на то, что инвариантность относительно дискретизации кажется свойством, которое представляет исключительно теоретический интерес, оно находит ряд приложений. Например, (пред)обработка изображений произвольного разрешения, последовательностей произвольной длины, графов и так далее.